Forex Arbitrage Detector

Project ki poori kahani, scratch se, aasaan Hinglish mein

Kiske liye: Chandan Kumar, Advanced Java (PBL) final project, faculty ko samjhane ke liye

Kya cover hai: CAP-1 se CAP-8 tak har cheez: algorithm, Swing app, MySQL, sockets, **Servlet + JSP portal (CAP-6)**, **Spring Boot + Hibernate/JPA + REST API (CAP-7)**, React website, aur ek **real-life feature: "Send money the cheapest way"**.

Kaise padhein: har screenshot par 1 2 3 orange number aur arrow hain. Screenshot ke neeche wahi number wali line batati hai ki woh cheez kya hai aur kyun hai.

Java nahi aati? Koi baat nahi. Har technical word pehli baar aate hi simple example se samjhaya gaya hai.

13 November 2026

1. Project ek nazar mein

Ek line mein: **computer har second currency rates dekhta hai aur aisa "loop" dhoondhta hai jismein paisa ghuma kar wapas aao to zyada ho jaye**. Phir woh loop screen par dikhata hai, MySQL mein save karta hai, aur alert bhejta hai.

Part A: Java Desktop App

Swing window (CAP-3). Algorithm, simulator, MySQL, sockets (CAP-1, CAP-5).

↓ same Java program ke andar

Part B: Web Portal + REST API

Browser mein `127.0.0.1:8080/portal`. Servlet + JSP (CAP-6), Spring Boot + Hibernate + REST (CAP-7).

↓ JSON data deta hai

Part C: React Website

Landing page + live desk. Vercel par online chal sakti hai. Same algorithm TypeScript mein.

Teen parts kyun?

- **Desktop app** course ke shuru ke CAPs ke liye hai: Java ka OOP, Collections, Swing, JDBC aur sockets.
- **Web portal** CAP-6 aur CAP-7 ke liye hai: website Java se kaise banti hai (Servlet, JSP) aur modern Spring Boot (Hibernate, REST).
- **React website** project ko sundar tarike se dikhane ke liye hai, aur koi bhi link khol kar try kar sakta hai.

> **Faculty ko ek line mein:** "Ma'am, ek hi Java Spring Boot program teen kaam karta hai: Swing desktop app, Servlet/JSP portal, aur REST API. React website us API ko use kar sakti hai, ya bina server ke bhi chal sakti hai."

2. Arbitrage kya hai? (example se)

Arbitrage = ek hi cheez ko alag-alag jagah ke rate ka fayda utha kar ghumao, aur wapas aao to **zyada** ho jaye.

Step	Kya kiya	Rate	Haath mein
Start			100 USD
1	USD se EUR	0.93	93 EUR
2	EUR se GBP	0.86	79.98 GBP
3	GBP se USD	1.26	**100.77 USD**

100 ke 100.77 ho gaye: **0.77% free profit**. Har conversion par thodi fee (0.05%) bhi lagti hai, isliye app sirf woh loop dikhata hai jo **fee ke baad bhi profit** de.

Computer yeh loop kaise dhoondhta hai? Bellman-Ford

1. **Map banao:** har currency ek "station" hai, har rate ek "line" (graph). Bilkul metro map jaisa.
2. **Multiply ko add mein badlo:** loop tab profit deta hai jab rates ka gunaa (multiply) 1 se bada ho. Bellman-Ford jod (add) ke saath kaam karta hai, isliye har rate ka `-log(rate)` lete hain. Ab "gunaa > 1" ka matlab ho jata hai "jod < 0", yaani **negative cycle**.
3. **Bellman-Ford chalao:** har round mein har line check karo ki kya isse chhota distance milta hai. n rounds ke baad bhi agar distance chhota hota ja raha hai, to pakka ek negative cycle hai, yaani **arbitrage mil gaya**.
4. **Loop nikalo:** "parent" pointers peeche follow karke loop ki currencies nikal lete hain, aur rates multiply karke profit % nikalta hai.

JAVA · ENGINE/GRAPHBUILDER.JAVA

```
double rateAfterFee = snapshot.rate(from, to) * (1 - fee);
edges.add(new Edge(from, to, -Math.log(rateAfterFee)));   // -log turns "product > 1" into "sum
< 0"
```

> **Rates kahan se aate hain?** Ek simulator (nakli market) har second rates thoda badalta hai, aur kabhi-kabhi ek rate jaan-boojh kar 0.6-1.2% galat kar deta hai, taaki demo mein loop dikhe. Asli market mein aisi galti milliseconds mein theek ho jati hai.

3. Code kahan hai? Folder structure (part by part)

Saara Java code `src/main/java/com/arbitrage/` mein hai. Har folder mein ek `package-info.java` file hai jo pehli line mein batati hai: "PART X (CAP-Y): yeh folder kya karta hai".

Folder	Part / CAP	Simple matlab
`model/`	Part 1 · CAP-1	Data ke dabbe: Currency, RateSnapshot, Opportunity (ek loop), OpportunityLeg (loop ka ek trade)
`engine/`	Part 1 · CAP-1	Dimaag: GraphBuilder, BellmanFordDetector, ProfitCalculator
`feed/`	Part 1 · CAP-1/5	Rates kahan se aayenge: simulator ya socket. TickScheduler har second detection chalata hai
`validation/` , `config/` , `service/`	Part 1 · CAP-1	Input check karna, settings, startup aur currency add/delete
`ui/`	Part 2 · CAP-3	Swing desktop window ki saari screens
`db/`	Part 3 · CAP-5	JDBC: MySQL se baat, PreparedStatement wali queries
`server/`	Part 4 · CAP-5	Doosra program: Rate Feed Server (TCP sockets)
`cap6/`	Part 5 · CAP-6	**Naya:** Servlets, Filters, Listener, login. JSP pages: `src/main/resources/META-INF/resources/WEB-INF/jsp/`
`cap7/` (+ `api/`)	Part 6 · CAP-7	**Naya:** Hibernate/JPA entities, repositories, services, REST controllers
`web/` (repo root par)	CAP-8 showcase	React website (TypeScript)
`src/test/java/`	sab parts	42 automatic tests (JUnit), plus 14 web tests
`docs/RUBRIC_GUIDE.md`	CAP-8	Rubric ki har line kis file mein hai, aur demo order

> Faculty puchhe "CAP-6 kahan hai?", to seedha `cap6/` folder kholo. "CAP-7 kahan hai?", to `cap7/` . Portal ke andar **Rubric map** page bhi hai jo yahi table live dikhata hai.

4. Chalana kaise hai

Ek hi command se sab start ho jata hai: MySQL, Feed Server, Java app (desktop + portal + REST) aur React website.

POWERSHELL / COMMAND PROMPT

```
cmd /c "C:\Users\CHANDAN\Pictures\experiments\java arbitage project\run-all.bat"
```

Kya khulega	Address	Note
Swing desktop app	apne aap window	CAP-1, 3, 5
Web portal (Servlet + JSP)	`http://127.0.0.1:8080/portal`	Login: **demo** / **demo1234**
REST API	`http://127.0.0.1:8080/api/v1/...`	Browser mein JSON dikhega
React website	`http://localhost:5173`	Vercel par bhi chal sakti hai

Tests chalana

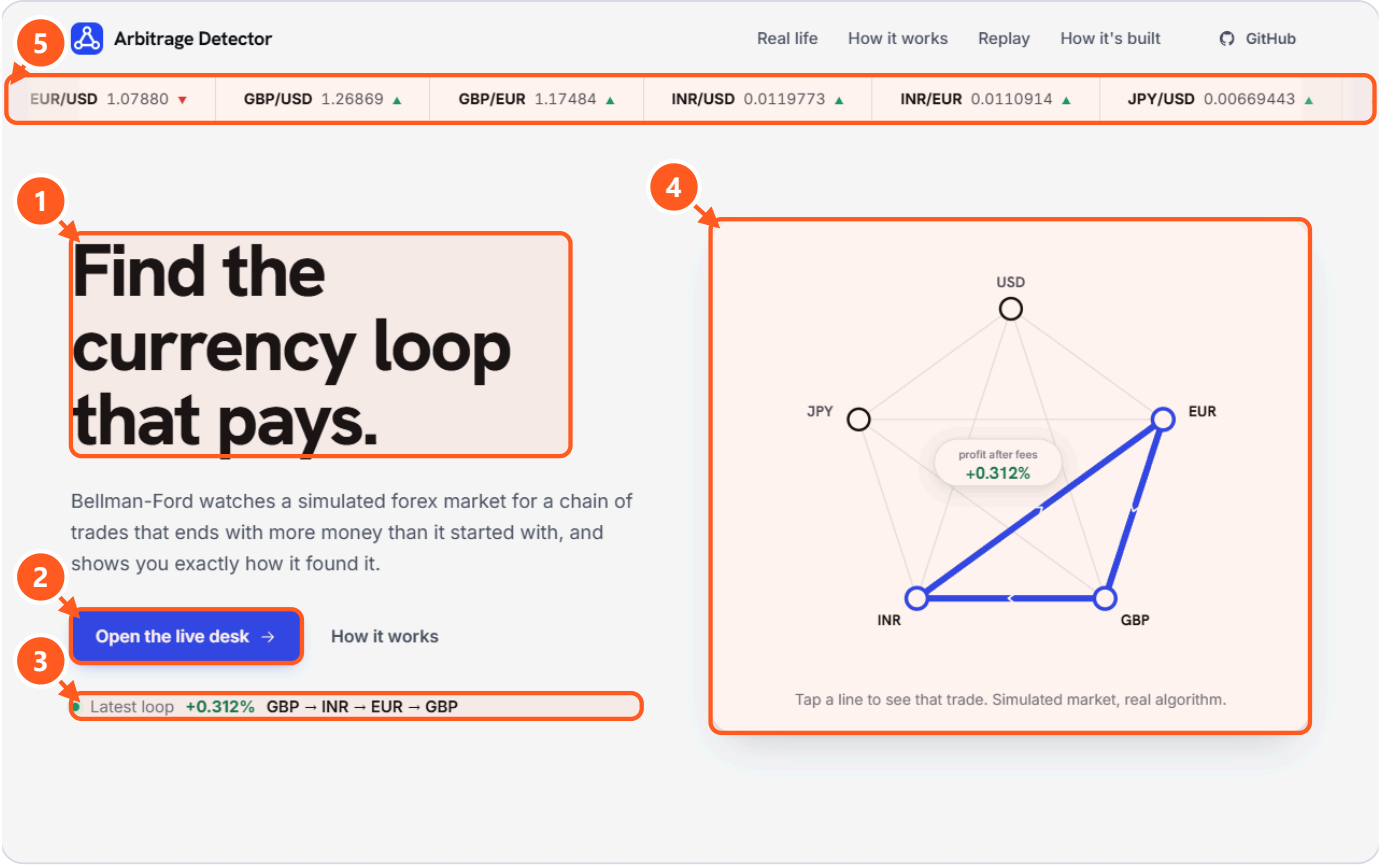
PROJECT FOLDER MEIN

```
.\mvnw.cmd test          # 42 Java tests  ->  BUILD SUCCESS
cd web ; npm test        # 14 web tests
```

> **Agar "Port 8080 already in use" aaye:** purani app band karo (uski window close karo), phir dobara chalao. **Agar "DB offline" aaye:** MySQL start nahi hua; `run-all.bat` dobara chalao.

5. React website: landing page

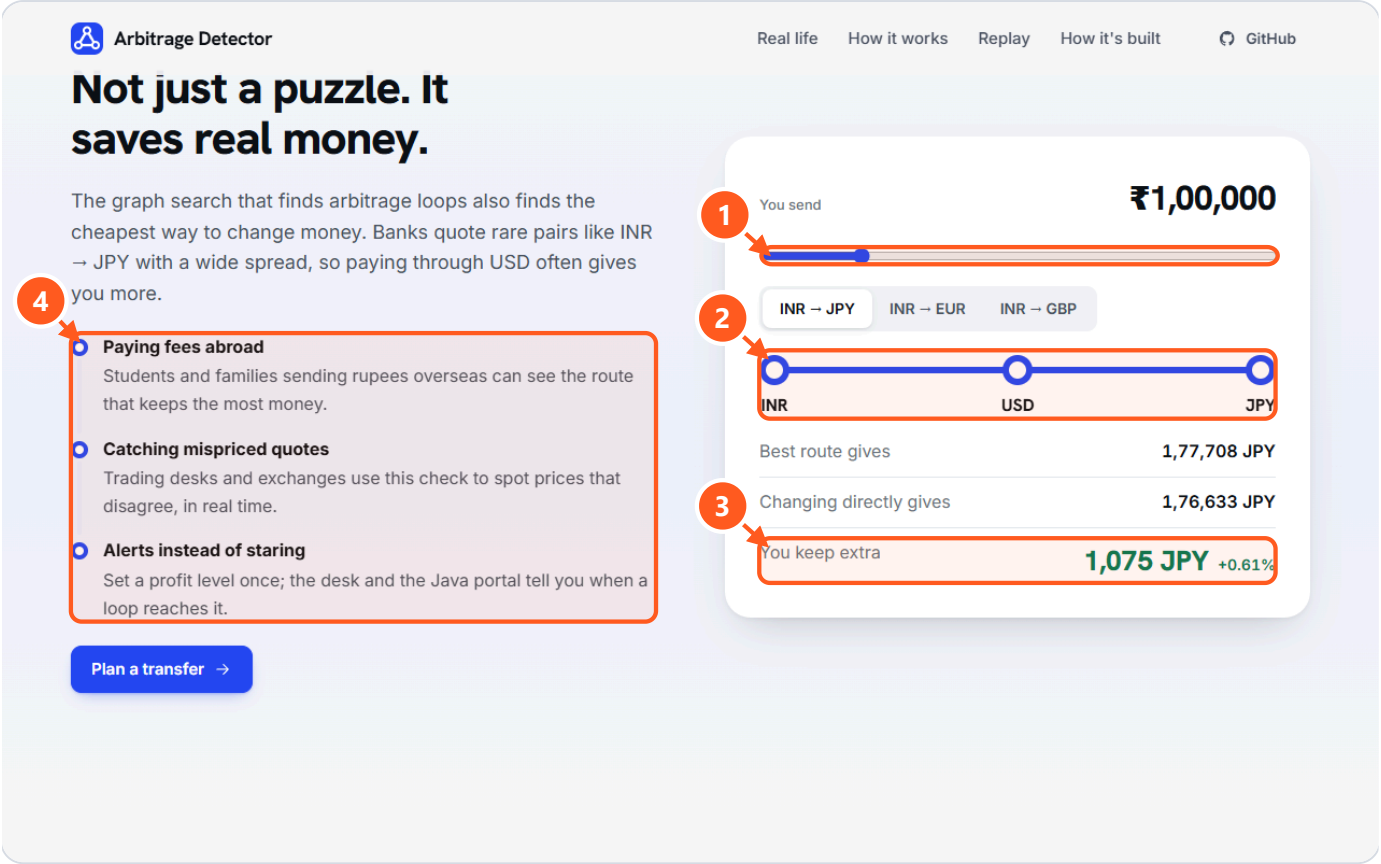
Yeh pehla page hai jo koi bhi link kholega. Saaf, light design, aur har cheez **live** chal rahi hai: rates har second badal rahe hain aur loops dhoondhe ja rahe hain.



1. **Headline:** project ek line mein: woh currency loop dhoondho jo paisa deta hai.
2. **Open the live desk:** yahan se poori app (desk) khulti hai.
3. **Latest loop:** abhi abhi mila loop aur uska profit %. Yeh har second update hota hai.
4. **Metro map:** currencies stations hain. Neeli line = profit wala loop. Kisi line par click karo to us trade ka rate dikhta hai.
5. **Rates ticker:** live exchange rates scroll hote hain. ▲ hara = rate badha, ▼ laal = rate gira.

6. Real life mein kya kaam aata hai? ("damn, yeh to sach mein problem solve karta hai")

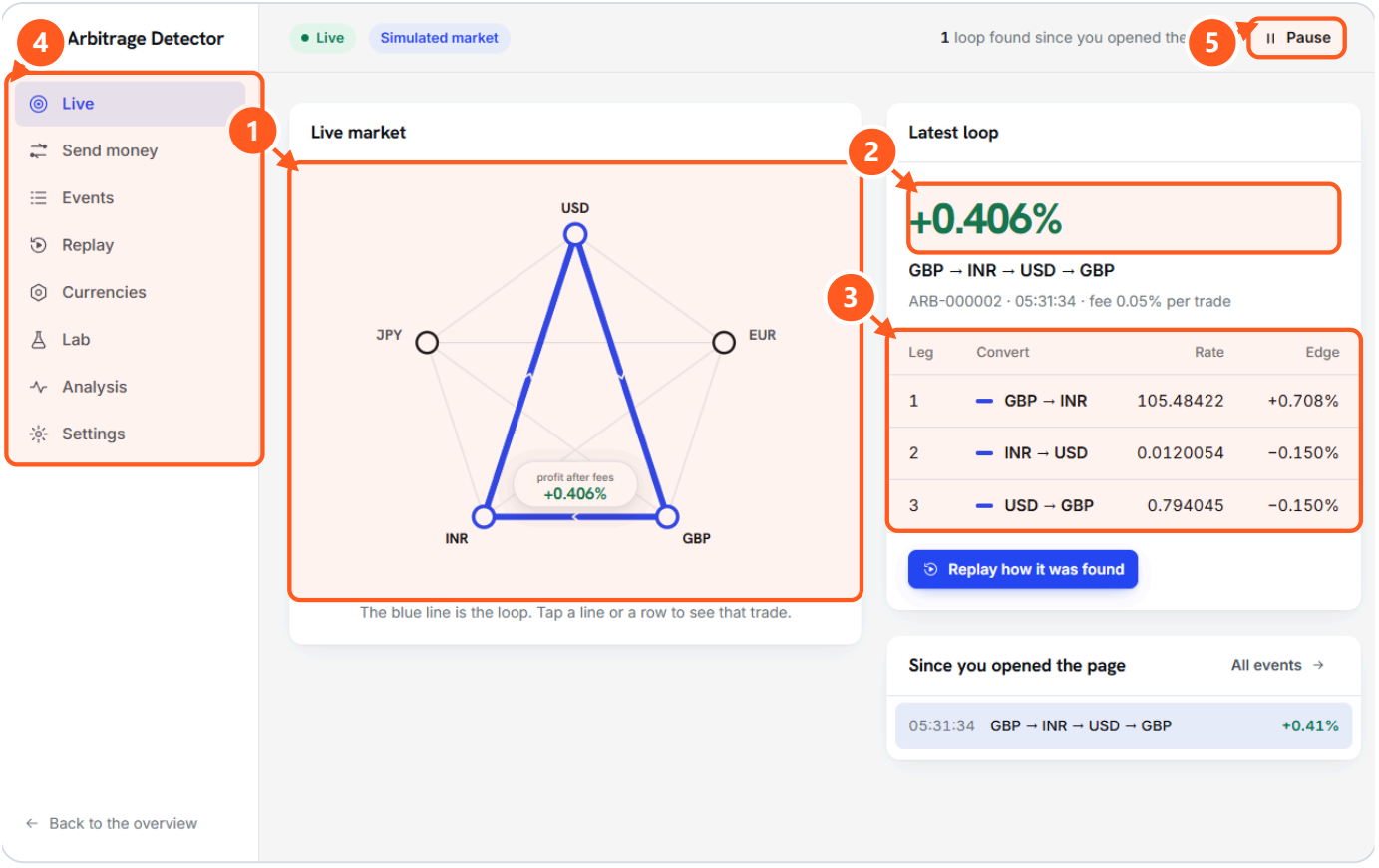
Sirf calculation nahi. Wahi graph search **paise bhejne ka sabse sasta raasta** dhoondhta hai. Bank rare pairs (jaise INR → JPY) par zyada spread (chhupi fees) lete hain, isliye **INR → USD → JPY** ghuma kar bhejna aksar sasta padta hai.



1. **Slider:** kitne rupaye bhejne hain (₹10,000 se ₹5,00,000). Slider hilao, sab turant update.
2. **Best route:** computer ne dhoondha ki INR seedha JPY mein badalne se behtar hai USD ke through jaana.
3. **You keep extra:** itne zyada yen milenge direct ke mukable. ₹1,00,000 par lagbhag 1,000+ yen bachte hain.
4. **Teen real uses:** (1) students/families jo bahar fees bhejte hain, (2) trading desks jo galat quotes pakadte hain, (3) alerts, taaki screen ghoorni na pade.

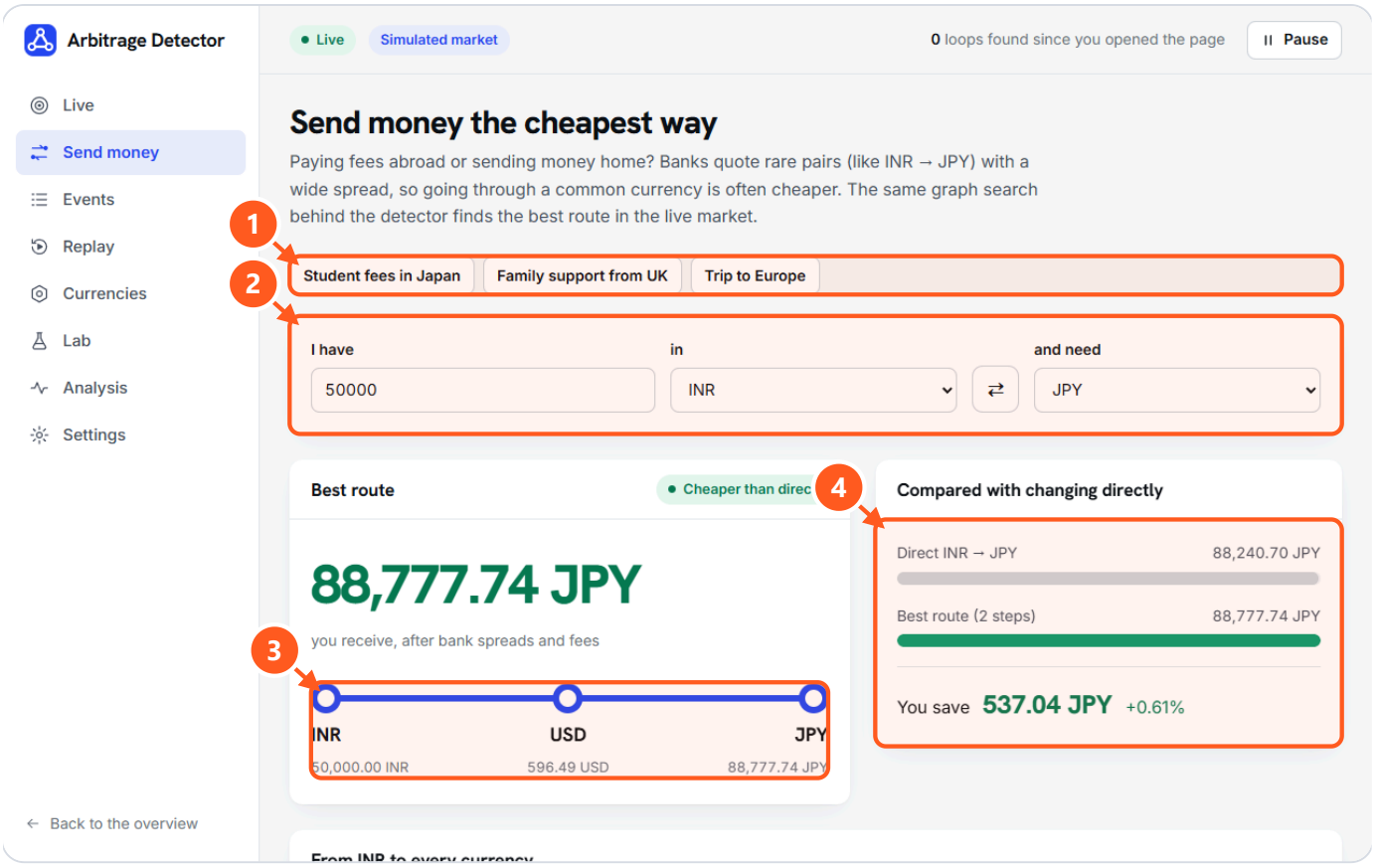
> **Kyun sach hai:** asli duniya mein bhi bade bank "vehicle currency" (zyadatar USD) ke through rare pairs ka rate banate hain. Humara model isi ko simple numbers se dikhata hai: major currency par 0.05% spread, baaki par 0.40%, aur jab kisi side USD na ho to 0.80% extra.

7. React desk: Live screen



1. **Live market map:** neeli line abhi ka profit wala loop hai. White pill mein "profit after fees".
2. **Profit:** bada hara number = is loop ka profit, sab fees ke baad.
3. **Legs table:** loop ke teeno trades. *Edge* column batata hai ki kaunsa trade "fair rate" se behtar tha. Jo + mein hai, wahi galat-priced rate hai jisne profit diya.
4. **Sidebar:** saari screens: Live, Send money, Events (history), Replay, Currencies, Lab, Analysis, Settings.
5. **Pause:** market rok kar aaram se dekh sakte ho (keyboard par Space bhi chalta hai).

8. React desk: Send money (real-life feature)

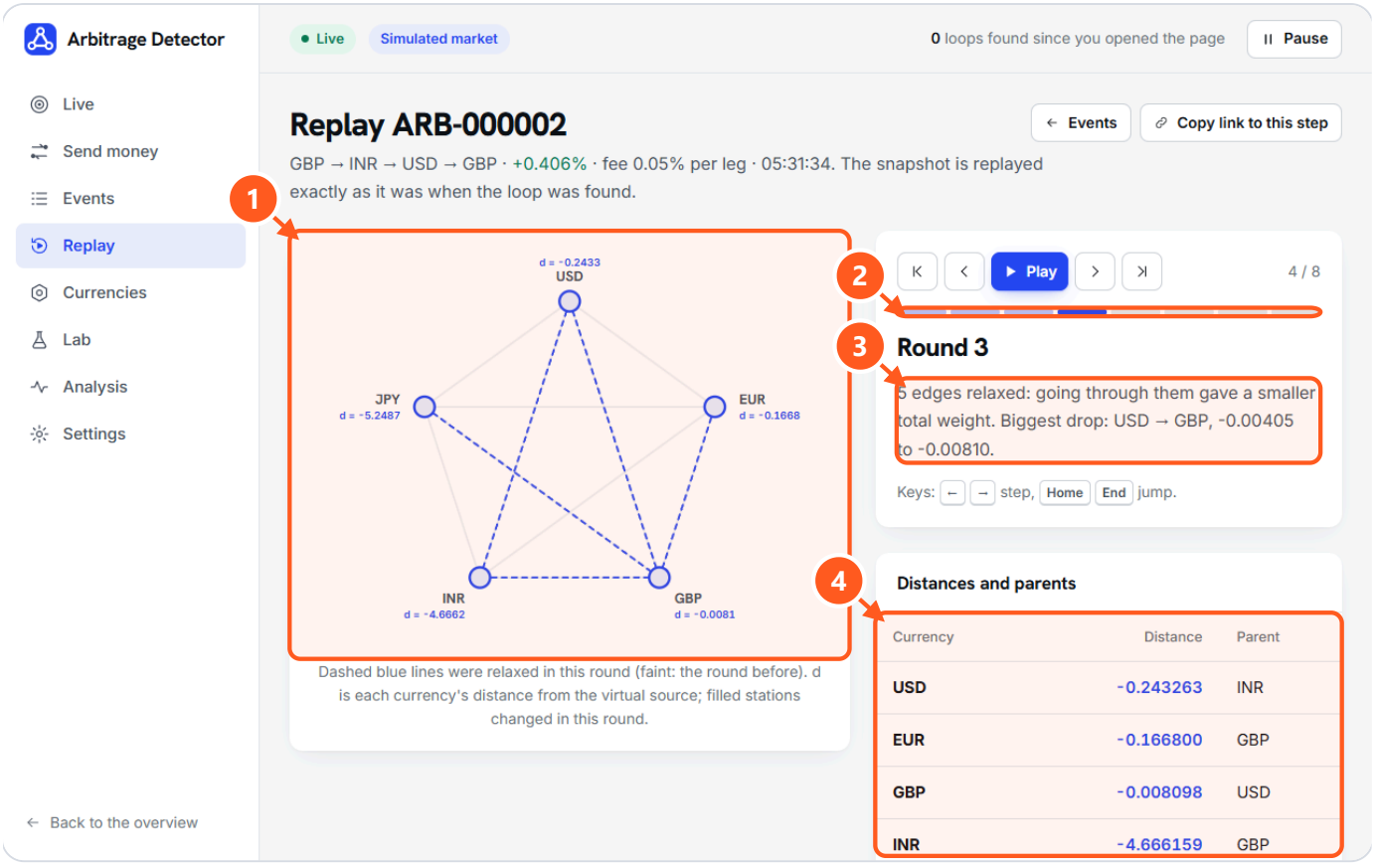


1. **Ready examples:** "Student fees in Japan", "Family support from UK", "Trip to Europe". Ek click mein form bhar jata hai.
2. **Form:** kitna paisa, kis currency mein hai, kis mein chahiye. ⇄ button dono currencies ulta-pulta kar deta hai.
3. **Route line:** har station ke neeche likha hai ki wahan pahunch kar kitna paisa hoga. Animation ke saath line banti hai.
4. **Comparison:** grey bar = seedha badalna, hari bar = best route. Neeche "You save" mein bachat.

Neeche ek table bhi hai: **INR se har currency** ka best route aur bachat, taaki ek nazar mein pata chale kahan sabse zyada fayda hai.

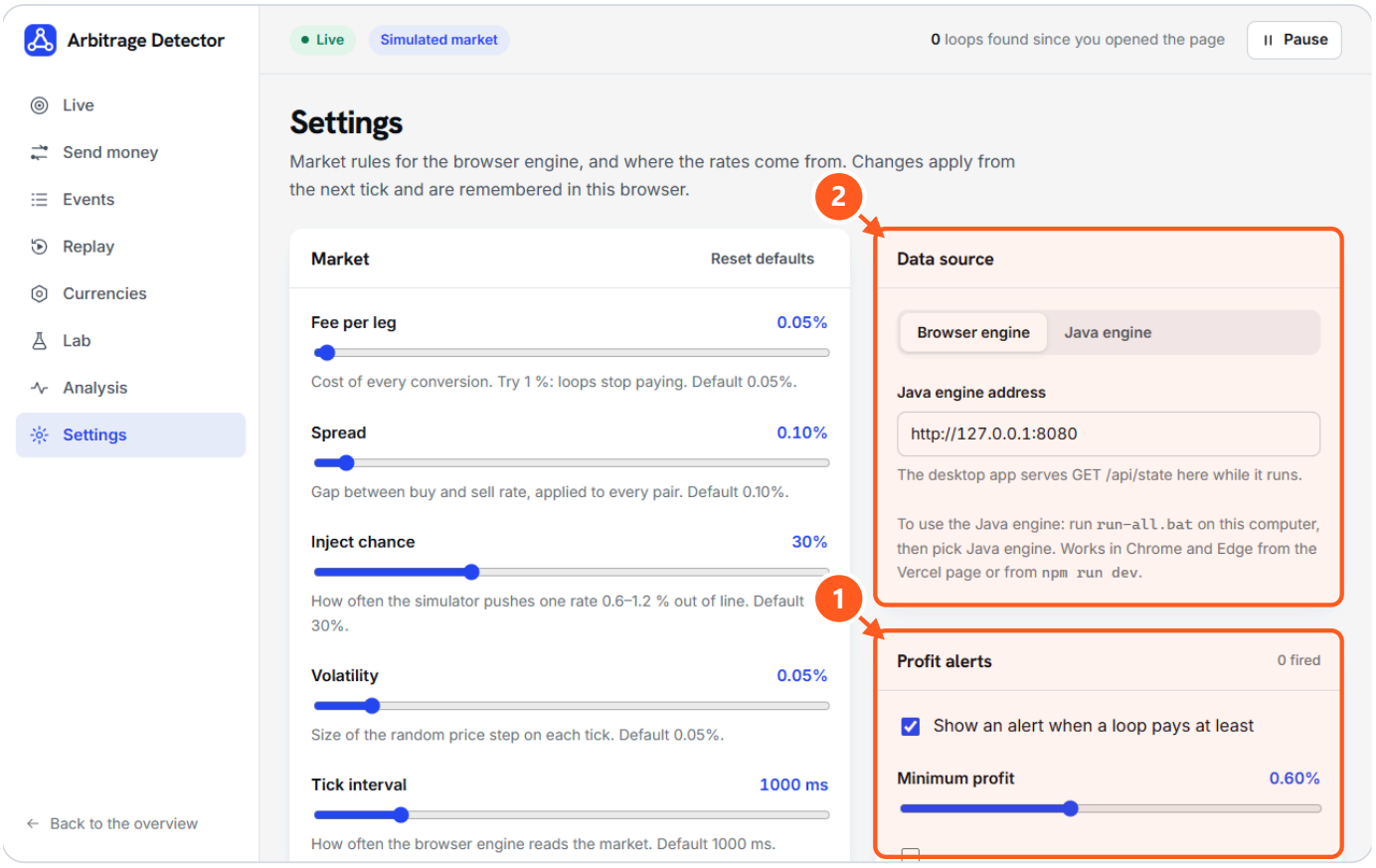
9. React desk: Replay (algorithm step by step) aur Alerts

Replay: ek purane loop ko dobara chala kar dekhna ki algorithm ne use kaise dhoondha



1. **Map par dotted lines:** is round mein Bellman-Ford ne kaunsi lines "relax" ki (yaani kahan chhota distance mila).
2. **Steps bar:** Virtual source → Round 1 → … → Check round → Walk back → Loop. Click karke kisi bhi step par jao, ya ← → keys.
3. **Explanation:** is step mein kya hua, simple English mein.
4. **Distances aur parents:** har currency ka distance aur uska parent. Yahi parent pointers aakhir mein loop nikalte hain.

Settings: alerts aur data source



1. **Profit alerts:** "jab koi loop itne % se zyada de, mujhe batao". Desk ke kone mein toast aata hai, aur tab background mein ho to browser notification bhi.
2. **Data source:** Browser engine (bina server, Vercel par) ya Java engine (laptop par chal rahi Spring Boot app se live data).

> Faculty ko dikhao: Settings mein **Java engine** chuno. Ab website ka data seedha tumhari Java app se aa raha hai (`GET /api/state` , CAP-7 REST).

10. CAP-6: Servlet, JSP aur MVC (scratch se)

Ek restaurant socho: **waiter** order leta hai, **kitchen** khana banati hai, aur khana **plate** mein saja kar aata hai.

Restaurant	Hamare project mein	Kaam
Waiter	**Servlet** (Controller)	Browser ki request leta hai (`doGet` = page dikhao, `doPost` = form ka data lo)
Kitchen	**Service / DAO** (Model)	Asli kaam: MySQL se data, calculation
Plate	**JSP** (View)	HTML page jismein data bhar kar user ko dikhate hain

Yahi **MVC** hai: Model-View-Controller. Servlet data ko *request attribute* mein rakhta hai aur JSP ko *forward* karta hai. JSP mein `${...}` (EL) se woh data padhte hain, aur `<c:forEach>` (JSTL) se table banti hai.

JAVA · CAP6/SERVLET/BASESERVLET.JAVA

```
protected void render(HttpServletRequest req, HttpServletResponse resp, String view) {
    req.getRequestDispatcher("/WEB-INF/jsp/" + view + ".jsp").forward(req, resp);   //
Controller -> View
}
```

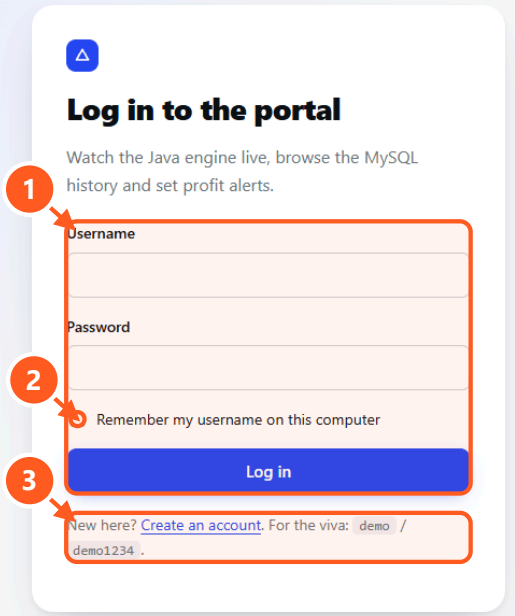
JSP · WEB-INF/JSP/HISTORY.JSP

```
<c:forEach items="${rows}" var="o">
  <tr><td>${o.id}</td><td><c:out value="${o.pathText}"/></td> ... </tr>
</c:forEach>
```

Baaki CAP-6 words, ek line mein

- **Session:** server ki yaaddasht ki "yeh user logged in hai". Login par user session mein rakha, logout par `invalidate()`.
- **Cookie:** browser mein chhoti si file. Humne "Remember my username" ke liye use kiya (password kabhi nahi).
- **Filter:** har page se pehle chalne wala guard. `AuthFilter` check karta hai ki login hai ya nahi, warna login page par bhej deta hai.
- **Listener:** events sunta hai. `PortalListener` ginta hai ki abhi kitne sessions khule hain.
- **@WebServlet / @WebFilter / @WebListener:** yeh annotations Tomcat ko batate hain ki kaunsa URL kis class ko jaye. `@ServletComponentScan` inhe dhoondh kar register karta hai.

11. Portal: Login (session, cookie, password hash)



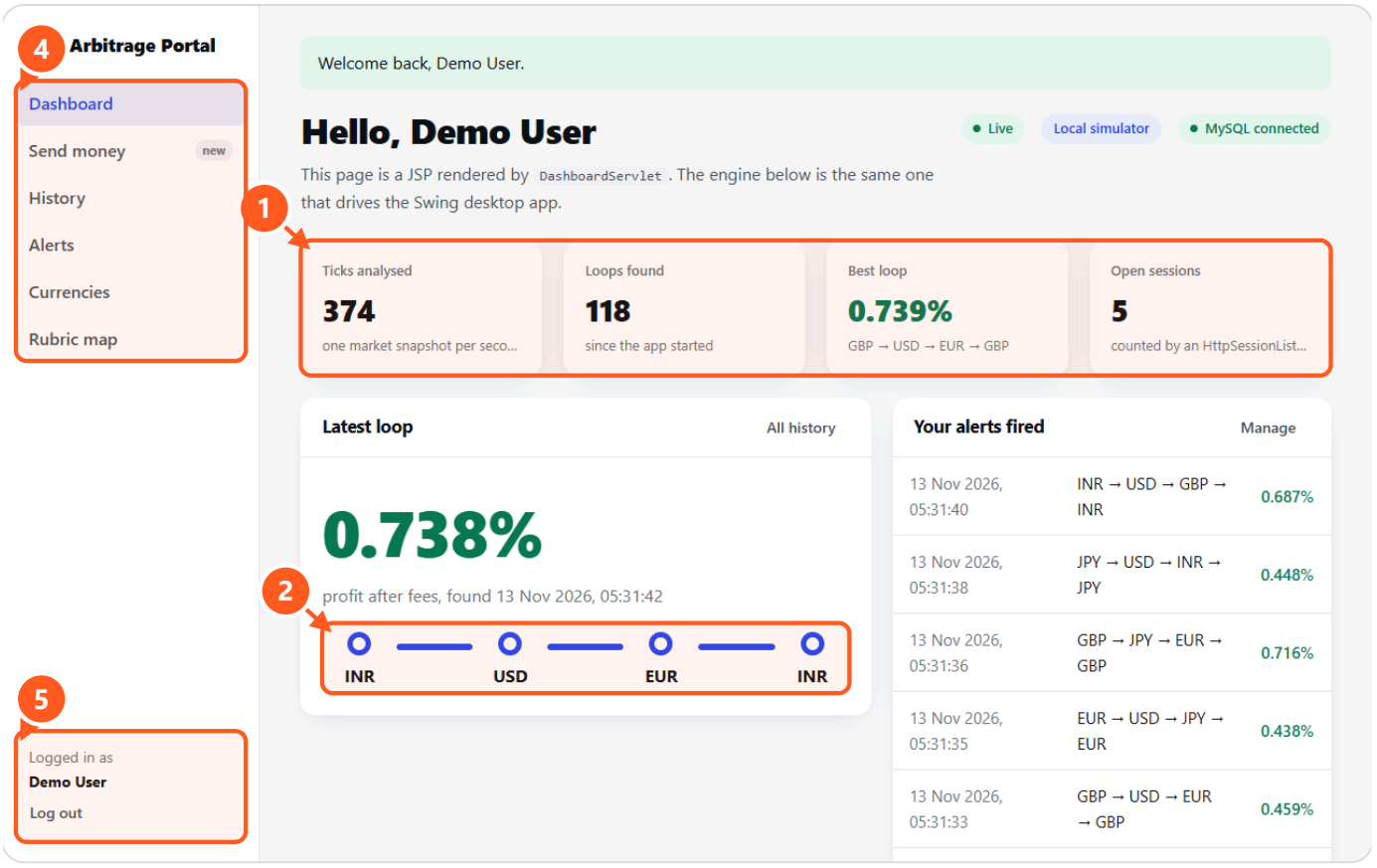
1. **Login form:** POST hota hai `LoginServlet.doPost()` par. Password check `PasswordHasher.matches()` se.
2. **Remember my username:** ek cookie set hoti hai (30 din). Agli baar username pehle se bhara milega.
3. **Demo account:** `demo / demo1234`, pehli baar app chalne par apne aap ban jata hai. Naya account "Create an account" se bhi bana sakte ho.

JAVA · CAP6/SERVLET/LOGINSERVLET.JAVA

```
HttpSession old = req.getSession(false);
if (old != null) old.invalidate();          // purana session khatam (session fixation se bachav)
HttpSession session = req.getSession(true);
session.setAttribute(USER, user);           // ab yeh user logged in hai
```

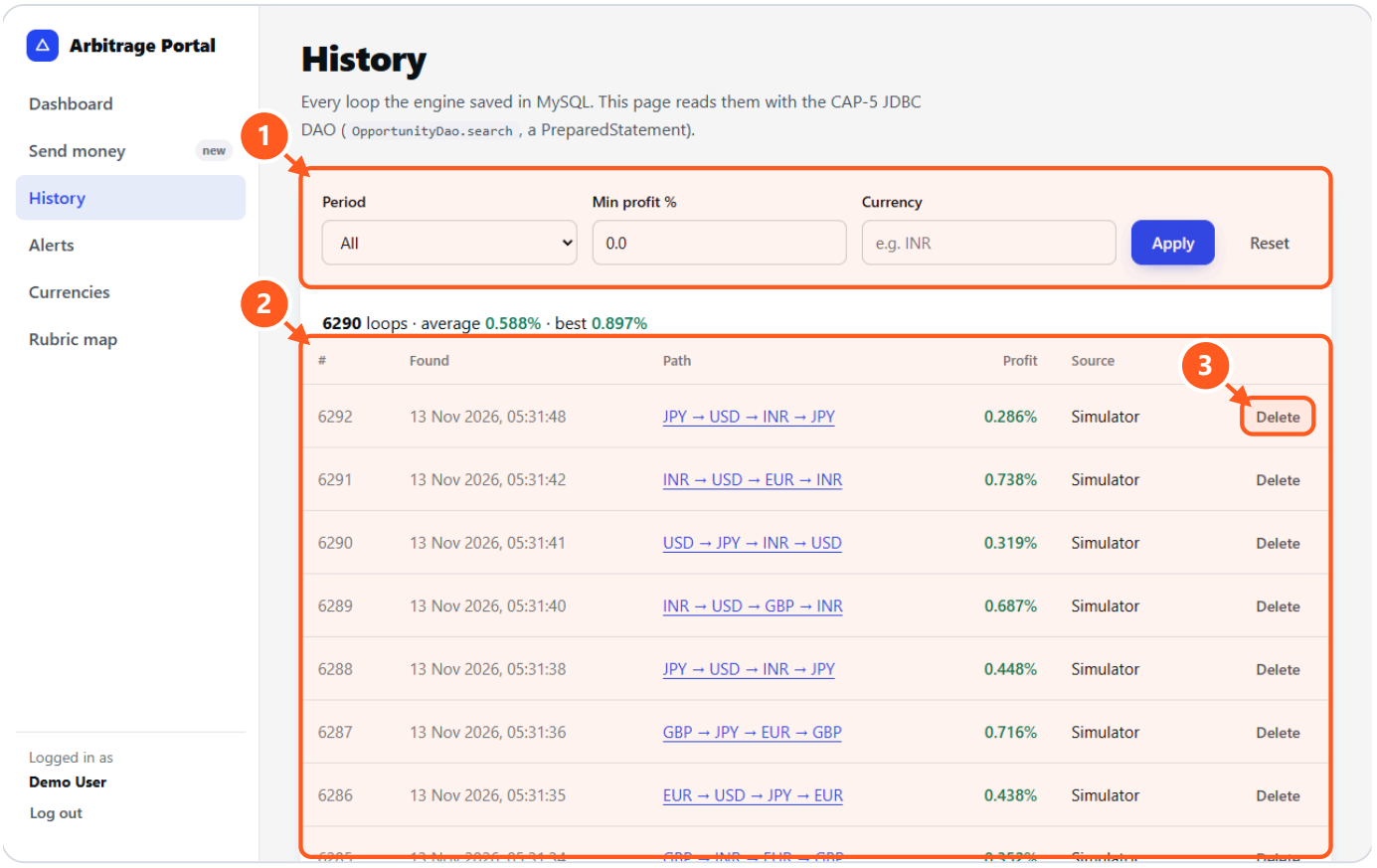
> **Password kabhi save nahi hota:** sirf uska **PBKDF2 hash** (ek tarfa taala) MySQL ke `app_user` table mein jata hai, har user ke liye alag random "salt" ke saath. Galat password par message: "Wrong username or password".

12. Portal: Dashboard (JSP + live update)



1. **Stat cards:** ticks, loops found, best loop, aur *Open sessions* (yeh number `HttpSessionListener` ginta hai).
2. **Latest loop:** metro line ki tarah loop. JSTL `<c:forTokens>` se path ke stations banaye.
3. **Recent loops table:** page reload kiye bina har 2 second update hoti hai: chhota JavaScript `/api/state` (REST) se naya data laata hai.
4. **Menu:** Dashboard, Send money, History, Alerts, Currencies, Rubric map. Sab pages `AuthFilter` ke peeche hain.
5. **Log out:** POST form + CSRF token. Session khatam, phir login page.

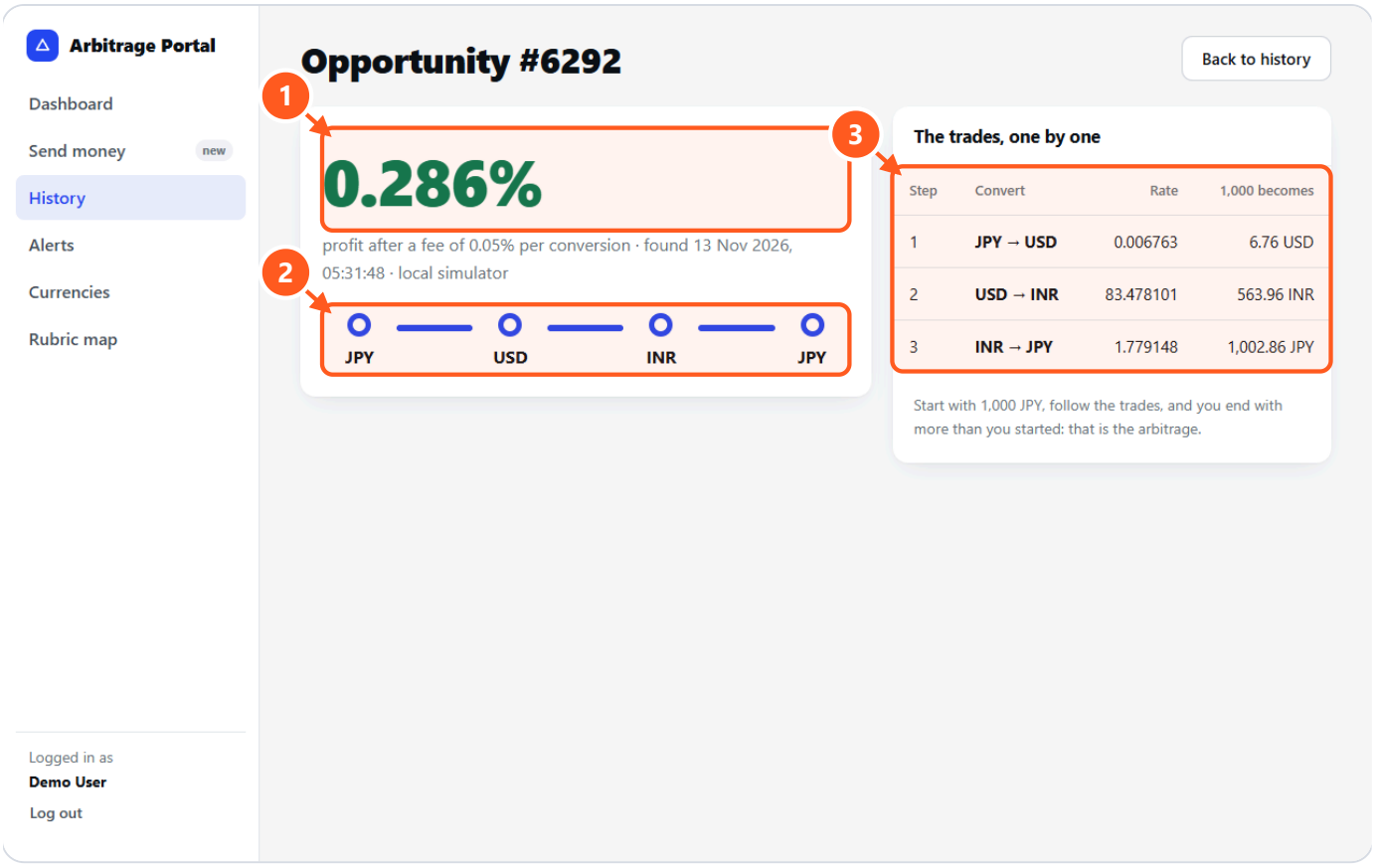
13. Portal: History (JDBC, filters, paging, delete)



1. **Filters:** Period, Min profit %, Currency. Yeh GET form hai, isliye filters URL mein aate hain (bookmark bhi kar sakte ho).
2. **Table:** MySQL se aata hai CAP-5 wale JDBC DAO se (`OpportunityDao.search` , PreparedStatement ke `?` placeholders ke saath, SQL injection se safe). 20 rows per page.
3. **Delete:** chhota POST form. Delete ke baad *redirect* hota hai (Post/Redirect/Get), taaki refresh dabane par dobara delete na ho. Upar hara message ek baar dikhta hai ("flash").

> **CSRF token kya hai?** Har form mein ek hidden random code hota hai jo sirf tumhare session ko pata hai. Koi doosri website tumhare naam se form submit nahi kar sakti, kyunki use woh code pata hi nahi.

14. Portal: ek Opportunity ki detail (Hibernate/JPA)



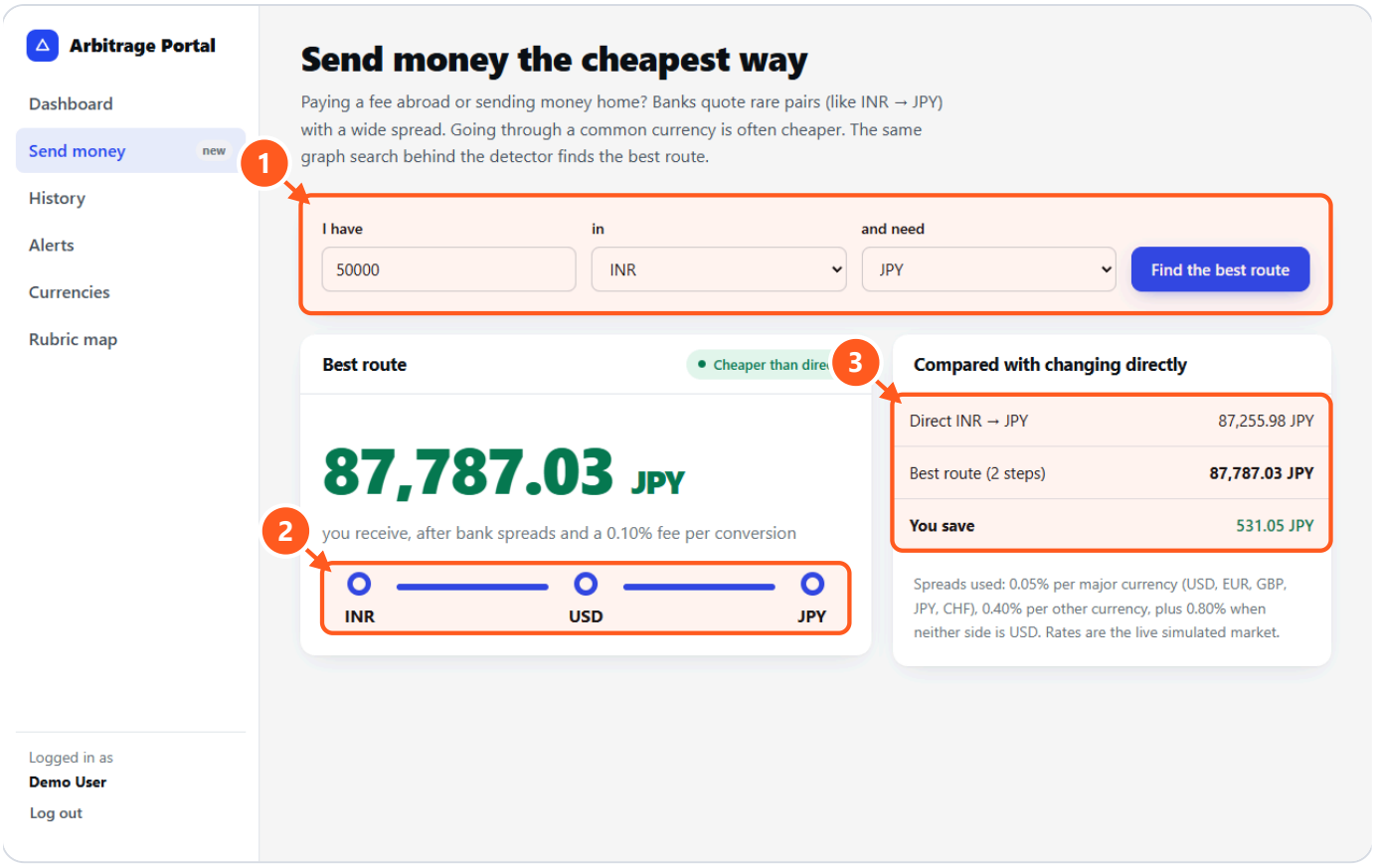
1. **Profit:** fee ke baad is loop ka profit.
2. **Metro line:** loop ki currencies, ek ke baad ek.
3. **The trades, one by one:** 1,000 se shuru karo aur har trade ke baad kitna bacha. Aakhir mein 1,000 se zyada, wahi arbitrage hai.

Yeh page **CAP-7 JPA** se data padhta hai: ek Opportunity ke kai Legs hain (**One-to-Many relationship**). Repository ki ek JPQL query `JOIN FETCH` se dono ek saath laati hai.

JAVA · CAP7/ENTITY/OPPORTUNITYENTITY.JAVA

```
@Entity @Table(name = "opportunity")
public class OpportunityEntity {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    @OneToMany(mappedBy = "opportunity", cascade = CascadeType.ALL) private
List<OpportunityLegEntity> legs;
}
```

15. Portal: Send money (real-life feature, Java version)

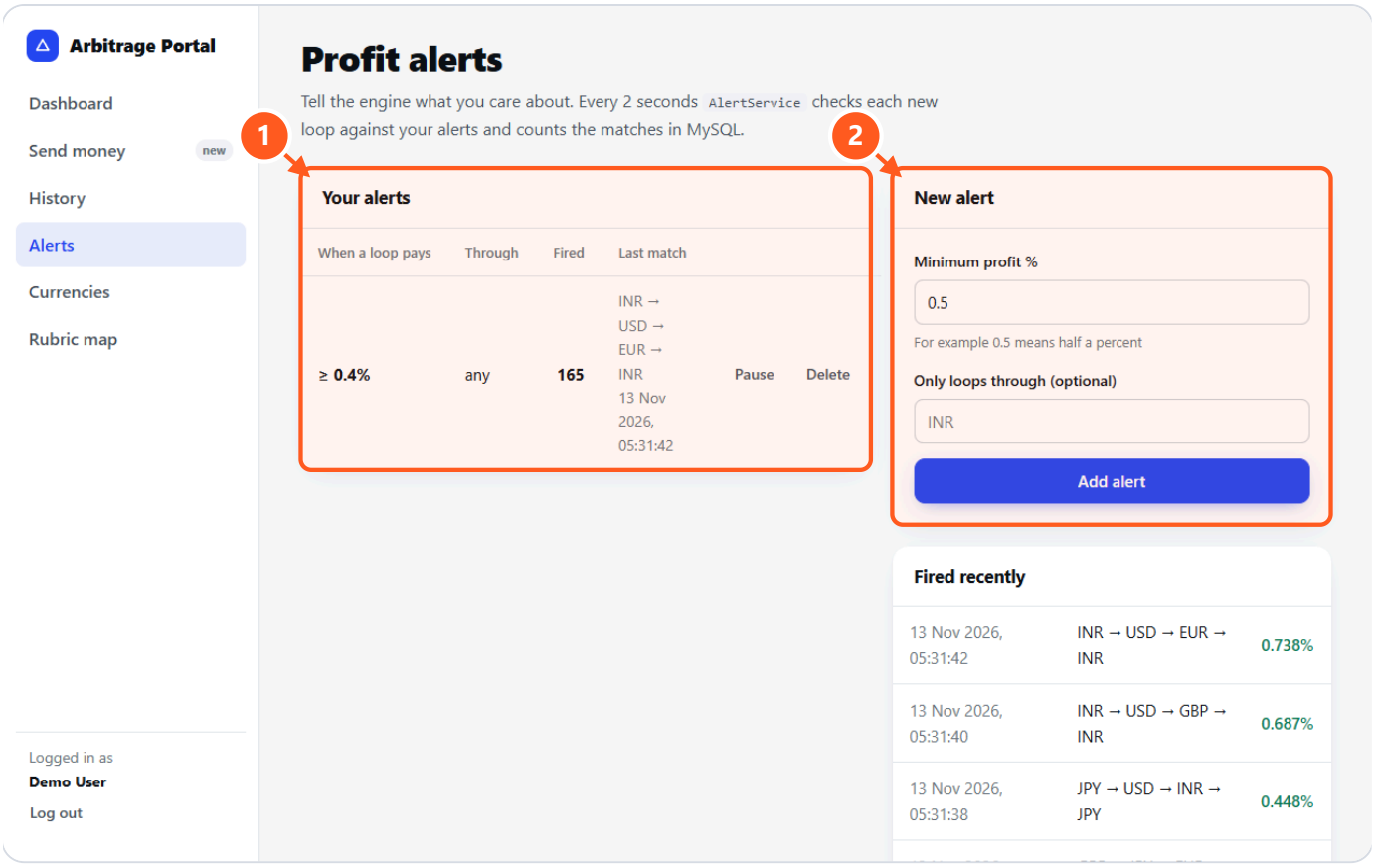


1. **Form:** kitna paisa, kis currency se, kis mein. `RouteServlet` isko `RouteService` ko deta hai.
2. **Best route:** INR → USD → JPY. Yeh shortest-path search (Bellman-Ford wala idea) se nikla.
3. **Comparison:** direct kitna milta, best route se kitna, aur **You save**.

JAVA · CAP7/SERVICE/ROUTESERVICE.JAVA

```java
// banks charge more for rare pairs; a pair without USD pays an extra 0.80 %
static double bankSpread(String from, String to) {
    double spread = spreadOf(from) + spreadOf(to);
    if (!from.equals("USD") && !to.equals("USD")) spread += CROSS_PENALTY;
    return spread;
}
```

16. Portal: Profit Alerts (JPA + @Scheduled)

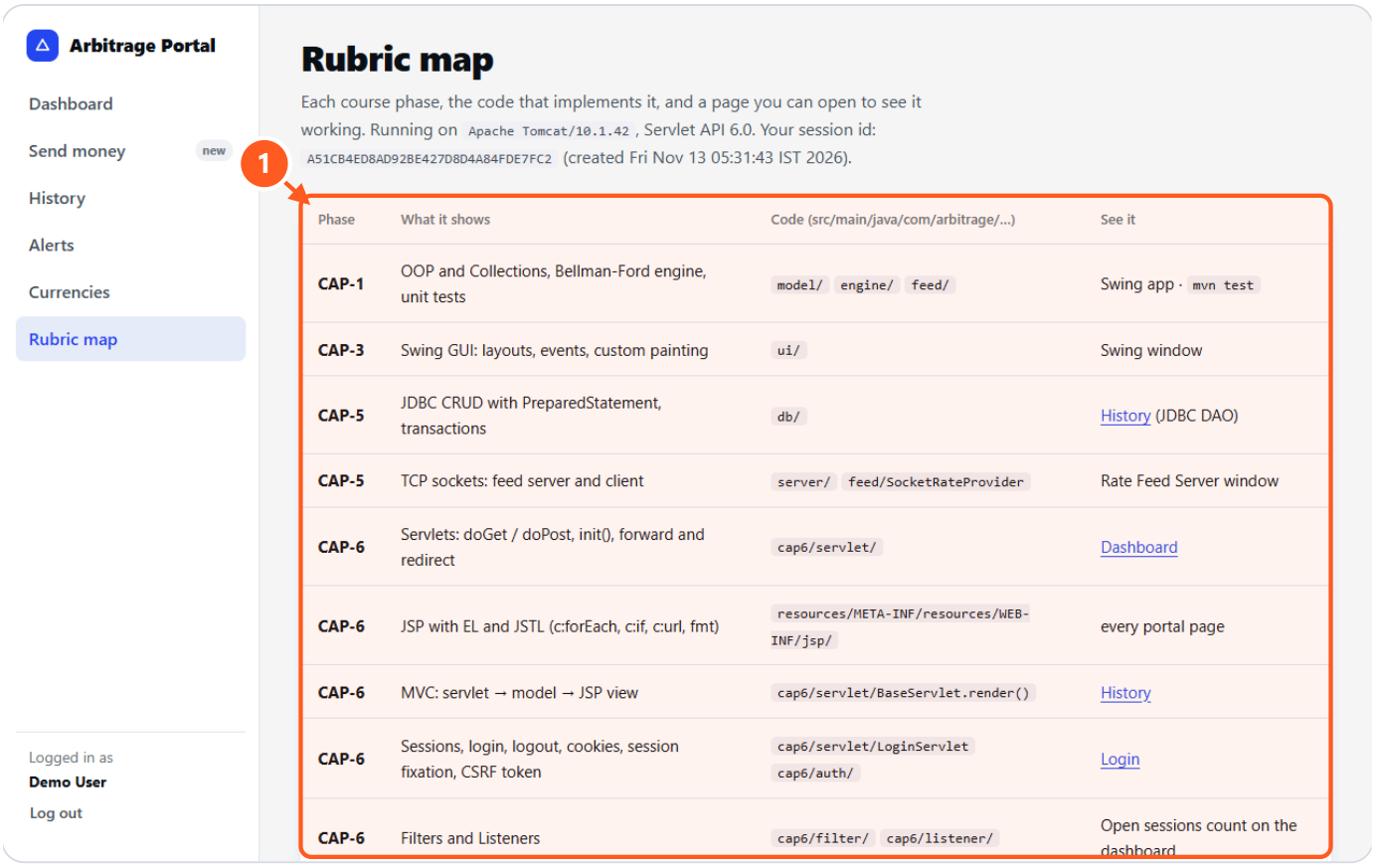


1. **Your alerts:** "jab loop ≥ 0.4% ho". Har alert MySQL ke `profit_alert` table mein JPA entity `ProfitAlert` ke roop mein save hai. *Fired* = kitni baar match hua.
2. **New alert + Fired recently:** naya alert banao; neeche dikhta hai ki abhi kaun se loops ne alert fire kiya.

`AlertService` mein ek method `@Scheduled(fixedDelay = 2000)` hai: Spring use har 2 second khud chalata hai. Woh naye loops ko active alerts se milata hai aur count badhata hai. `@Transactional` ka matlab: sab changes ek saath save, ya koi bhi nahi.

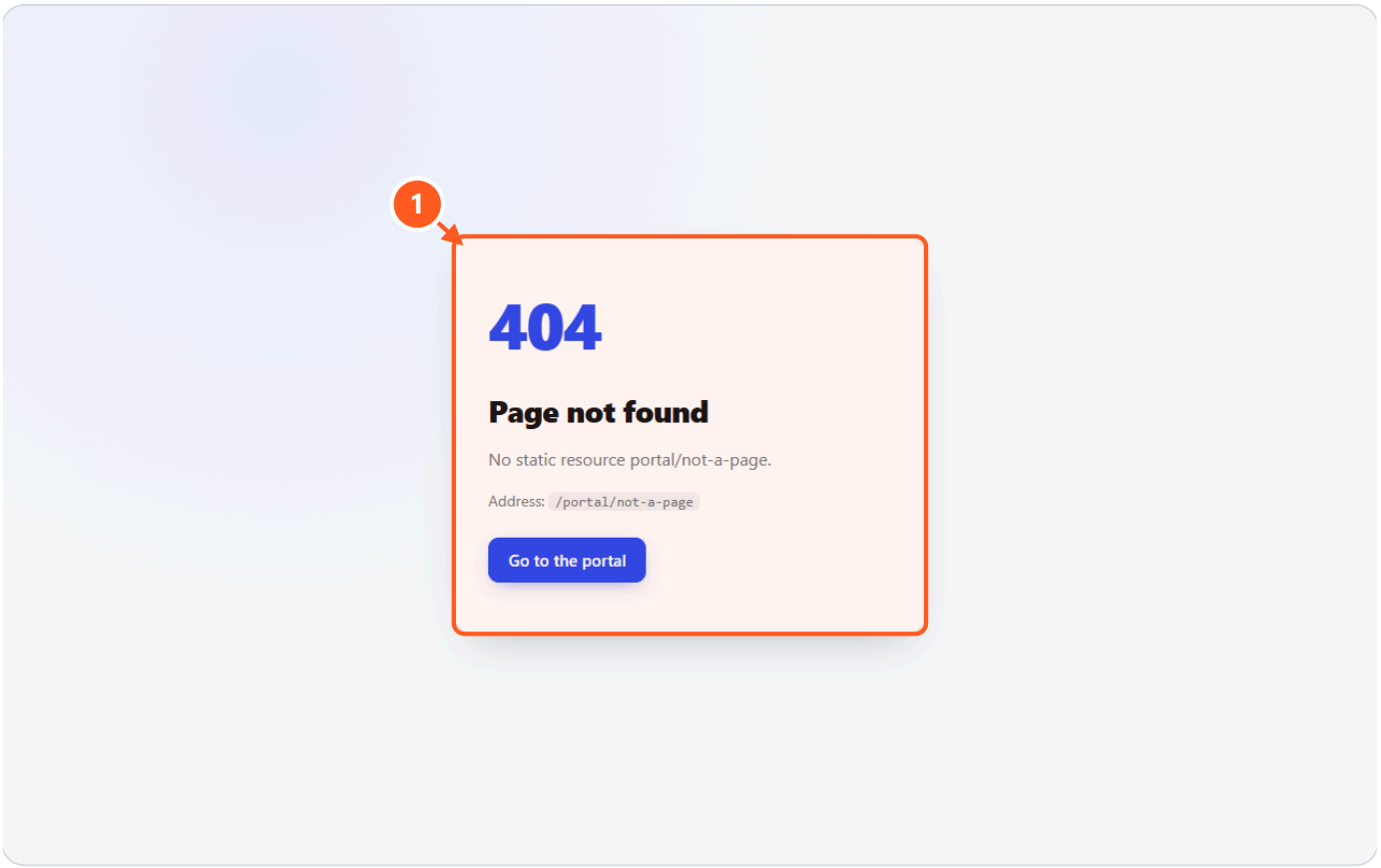
> **Real life:** trader ka "price alert" jaisa. Screen ghoorne ki zaroorat nahi; jab kaam ki cheez ho, system bata deta hai.

17. Portal: Rubric map aur Error page



1 **Rubric map:** har CAP, uska code folder, aur ek link jahan woh cheez live dikhti hai. Viva mein yahi page khol kar chalo.

Error page



1 **404 page:** galat address par sundar error page (`ErrorServlet` + `error.jsp`). Yeh `ErrorPagesConfig` mein register hai, bilkul web.xml ke <error-page> jaisa. API ke galat address par JSON error aata hai.

18. CAP-7: Spring Boot, Hibernate/JPA, REST (scratch se)

- **Spring Boot:** ek framework jo objects khud bana kar jodta hai (*dependency injection*) aur Tomcat server andar hi chala deta hai. Humein bas classes likhni hain.
- **Hibernate / JPA:** SQL likhne ki jagah Java class likho. `@Entity` class = MySQL table, ek object = ek row. Hibernate SQL khud banata hai.
- **Repository:** ek interface. `findByActiveTrue()` jaisa naam likho aur Spring uski query khud bana deta hai (*derived query*). Mushkil wali ke liye `@Query` (JPQL).
- **Service + @Transactional:** business logic. Transaction = sab ya kuch nahi.
- **REST API:** URL + HTTP method se data lena-dena, JSON mein. GET = padho, POST = banao, PUT = badlo, PATCH = thoda badlo, DELETE = hatao.
- **DTO + Validation:** JSON ke liye chhoti record classes. `@NotBlank`, `@Pattern`, `@Positive` galat input ko 400 error ke saath rok dete hain.
- **@RestControllerAdvice:** ek jagah jo har error ko saaf JSON mein badalti hai (404, 409, 400, 503).

JAVA · CAP7/REST/CURRENCYCONTROLLER.JAVA

```java
@PostMapping
public ResponseEntity<CurrencyDto> create(@Valid @RequestBody CurrencyRequest request) {
    CurrencyDto created = service.create(request);
    return ResponseEntity.created(location).body(created);     // 201 Created + Location header
}
```

Status code	Matlab	Kab aata hai
200 OK	sab theek	GET sahi chala
201 Created	naya bana	POST currency / alert
204 No Content	ho gaya, kuch lautane ko nahi	DELETE
400 Bad Request	input galat	code "aud" chhote letters mein
404 Not Found	cheez hai hi nahi	currency XYZ
409 Conflict	data se takraav	USD delete karna, ya duplicate code
503 Unavailable	MySQL band hai	database offline

19. REST API ke asli jawab (app se live liye gaye)

Yeh JSON abhi chal rahi app se fetch kiye gaye hain. Browser mein yahi address khol kar faculty ko dikha sakte ho.

GET /api/v1/opportunities/summary — 200


```
{
  "period": "all",
  "count": 6291,
  "averageProfit": 0.5874412407,
  "bestProfit": 0.896578
}
```


GET /api/v1/route?from=INR&to=JPY&amount=50000 — 200


```
{
  "from": "INR",
  "to": "JPY",
  "amount": 50000,
  "path": [
    "INR",
    "USD",
    "JPY"
  ],
  "received": 87737.44913450857,
  "directReceived": 87206.7029146736,
  "saving": 530.7462198349676,
  "stepRates": [
    0.011905088036933876,
    147.39487664822866
  ]
}
```


GET /api/v1/currencies/XYZ — 404

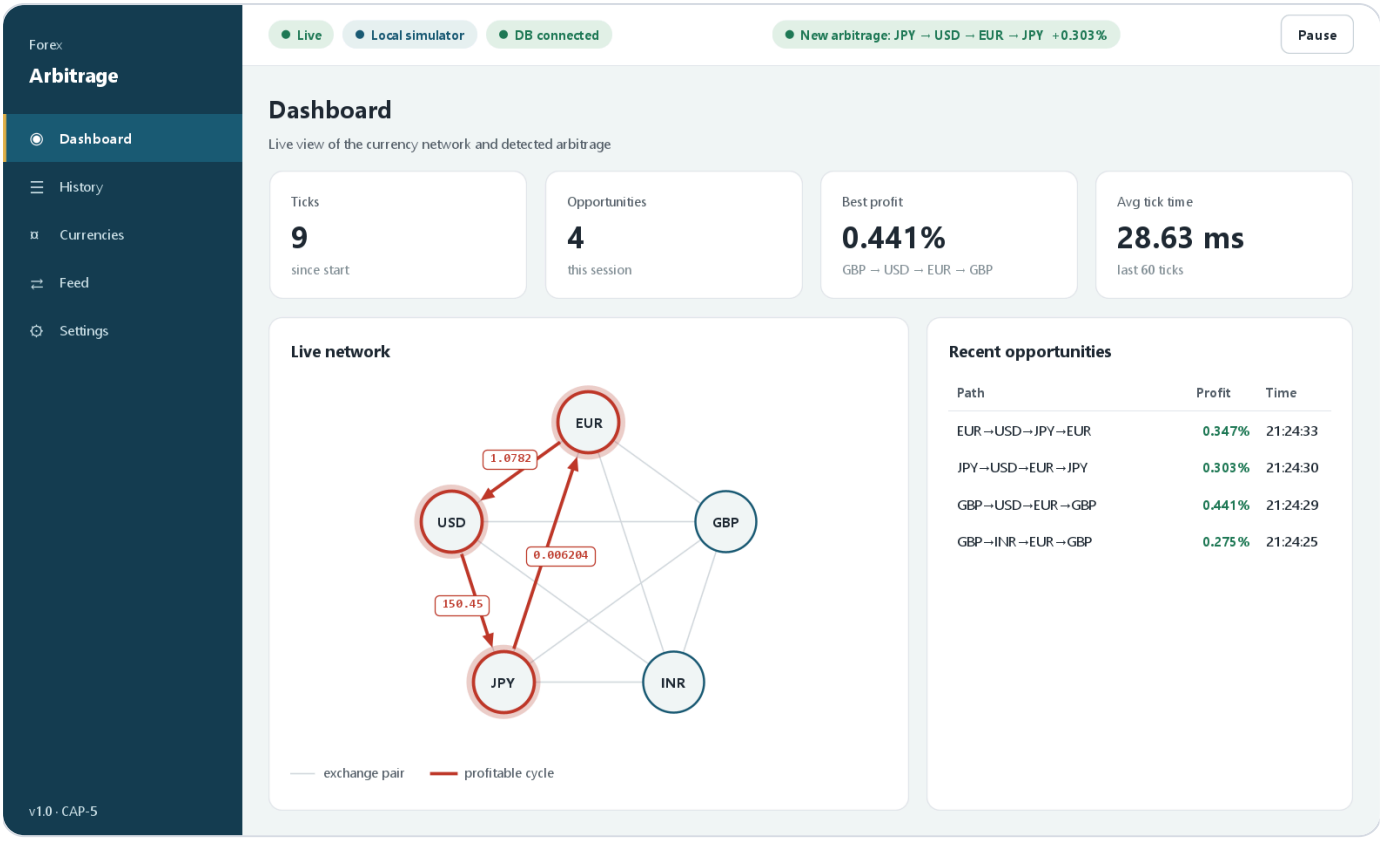

```
{
  "timestamp": "2026-11-13T00:01:56.276834300Z",
  "status": 404,
  "error": "Not Found",
  "message": "Currency XYZ does not exist",
  "path": "/api/v1/currencies/XYZ",
  "fieldErrors": {}
}
```


POST /api/v1/currencies — 400


```
{
  "timestamp": "2026-11-13T00:01:56.295402700Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Some fields are not valid",
  "path": "/api/v1/currencies",
  "fieldErrors": {
    "code": "must be 3 uppercase letters, like AUD",
    "name": "size must be between 2 and 40",
    "usdPrice": "must be greater than 0"
  }
}
```


Note: **400** wale jawab mein har galat field ka apna message hai. Yahi Bean Validation ka kamaal hai.

20. Purane CAPs ek nazar mein: Swing desktop app (CAP-1, 3, 5)



- **CAP-1:** records (model), Collections (ArrayList, HashMap, HashSet, ArrayDeque, CopyOnWriteArrayList), Bellman-Ford engine, JUnit tests.
- **CAP-3:** Swing: CardLayout (screens badalna), GridBagLayout, BorderLayout, event listeners, khud banaya network graph.
- **CAP-5:** JDBC + MySQL (PreparedStatement, transaction: ek loop aur uske legs ek saath save), aur TCP sockets: Rate Feed Server sab desktops ko har second rates bhejta hai.

> Teeno parts ek hi engine use karte hain. Swing mein dikhne wala loop wahi hai jo portal aur website mein dikhta hai.

21. Viva ke possible sawaal (short jawab)

Servlet aur JSP mein kya fark hai?

Servlet Java class hai jo request handle karti hai (logic). JSP HTML page hai jismein data dikhate hain (view). JSP andar se servlet mein hi compile hota hai.

forward aur redirect mein fark?

forward server ke andar hi doosre page ko bhejta hai (URL nahi badalta). redirect browser ko naya URL kholne ko kehta hai. Form POST ke baad redirect karte hain (PRG).

Session kaise kaam karta hai?

Server user ke liye ek session banata hai aur browser ko ek cookie (JSESSIONID) deta hai. Har request par woh cookie aati hai, to server pehchaan leta hai.

Filter kyun?

Har page par login check alag-alag likhne ki jagah ek AuthFilter sabse pehle chalta hai.

JDBC aur JPA/Hibernate mein fark?

JDBC mein SQL khud likhte hain (CAP-5, `db/`). JPA mein class (entity) likhte hain aur Hibernate SQL banata hai (CAP-7, `cap7/`). Project mein dono same tables par hain.

@OneToMany kahan hai?

OpportunityEntity → OpportunityLegEntity. Ek loop ke kai trades. Legs ka primary key do columns ka hai, isliye @IdClass.

REST mein PUT aur PATCH?

PUT poori cheez badalta hai (currency ka naam, price, active). PATCH sirf ek hissa (alert on/off).

Password kaise safe hai?

PBKDF2 hash + random salt. Database mein asli password kabhi nahi.

Bellman-Ford hi kyun, Dijkstra kyun nahi?

Dijkstra negative weights handle nahi karta. Bellman-Ford karta hai aur negative cycle (= arbitrage) bhi pakadta hai.

Real life mein kya use?

(1) Paise bahar bhejne ka sasta raasta, (2) trading desks par galat quotes pakadna, (3) profit alerts.

MySQL band ho to?

App phir bhi chalti hai: "DB offline" dikhata hai, detection memory mein chalta hai. REST 503 deta hai.

22. Faculty ke saamne demo (10 minute ka order)

1. `.\mvnw.cmd test` chalao → **42 tests pass**. "Ma'am, algorithm aur API ke tests hain."
2. `run-all.bat` → Swing window: neeli/laal line mein loop. Bellman-Ford samjhao (page 2).
3. Browser: `127.0.0.1:8080/portal` → **demo / demo1234** login. Dashboard ke numbers khud update hote dikhao.
4. History → filter lagao → ek opportunity kholo (JPA, One-to-Many). Ek delete karke dikhao (PRG + flash message).
5. **Send money** → ₹50,000 INR → JPY. "Ma'am, USD ke through bhejne se itne yen bachte hain." (real-life)
6. Alerts → "0.5%" ka alert banao → kuch second mein "Fired" count badhega.
7. REST: `/api/v1/opportunities/summary` aur `/api/v1/currencies/XYZ` (404 JSON) kholo.
8. Rubric map page → har CAP ka code folder dikhao. VS Code mein `cap6/` aur `cap7/` kholo.
9. React website: landing ka slider ghumao, desk mein Replay chalao.
10. Log out → Back button dabao → page nahi khulega (filter + no-store).

> **Yaad rakhne layak ek line:** "Yeh project rates ko graph bana kar Bellman-Ford se arbitrage loops dhoondhta hai, unhe Swing, Servlet/JSP portal aur React website par dikhata hai, Hibernate se MySQL mein rakhta hai, REST API se data deta hai, aur real life mein paise bhejne ka sabse sasta raasta batata hai."